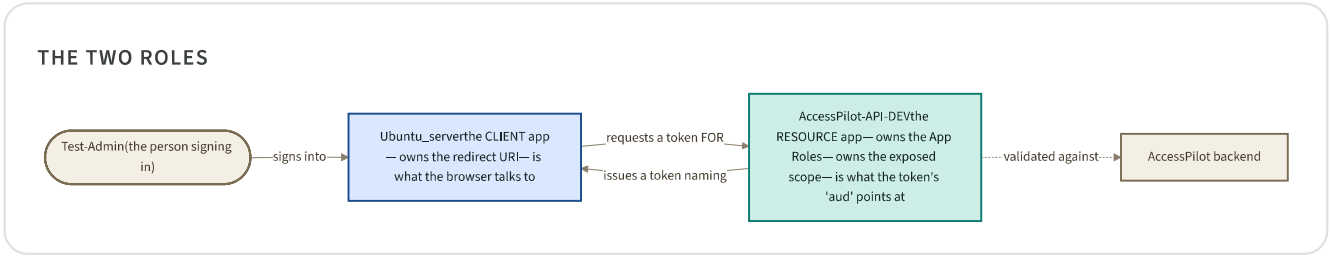
ACCESSPILOT · CONCEPT

Why Ubuntu_server needed permission to use AccessPilot-API-DEV

Two Entra app registrations were involved in your sign-in, playing two different jobs. Nothing here needs code — it's entirely about what's configured in Entra and in AccessPilot's own settings.

The two jobs

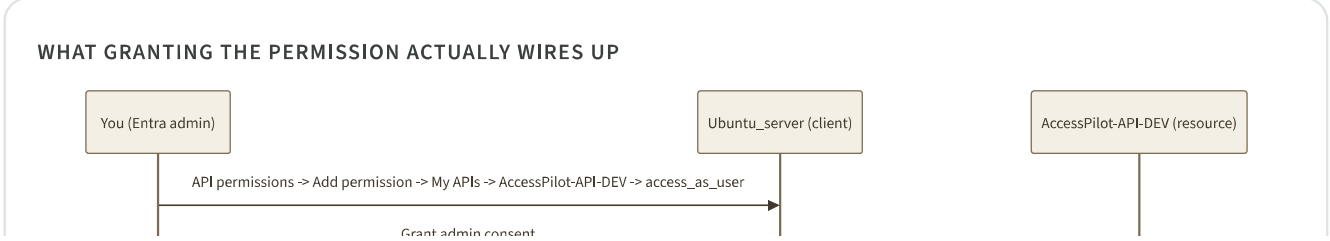
Every sign-in needs a **client** — the app the user actually clicks "sign in" on — and a **resource** — the app the access token is actually *for*. They're often the same app registration. In your setup they ended up being two different ones:



Think of **Ubuntu_server** as the *front door* the user walks through, and **AccessPilot-API-DEV** as the *vault* the app actually needs a key to. Walking through the front door doesn't automatically hand you a key to the vault — someone has to explicitly say "the front door is allowed to issue vault keys," and separately, someone has to decide *which people* get a key at all.

Why a permission had to be granted at all

By default, Entra has no reason to connect the two. Ubuntu_server signing a user in doesn't imply anything about AccessPilot-API-DEV. You have to explicitly tell Entra: *"Ubuntu_server is allowed to request a token for AccessPilot-API-DEV's* `access_as_user` *scope."* That's exactly what **API permissions → Add a permission → My APIs → AccessPilot-API-DEV** did, on the Ubuntu_server side.





Without it: Entra doesn't refuse to sign the user in — it just quietly hands back a token for a different, default resource (Microsoft Graph) instead. Sign-in looks completely fine. Every call into AccessPilot's own API then fails, with nothing in the UI pointing at why.

What that actually looked like, side by side

✕ Permission missing — silent fallback

```
{
  "aud": "00000003-0000-...",
  "scp": "openid profile email"
  // no "roles" at all
}
```

That GUID is Microsoft Graph's own fixed app ID — not yours.

✓ Permission granted + consented

```
{
  "aud": "3282c855-...-9408127bdf08",
  "scp": "access_as_user",
  "roles": ["AccessPilot.User"]
}
```

aud now points straight at AccessPilot-API-DEV.

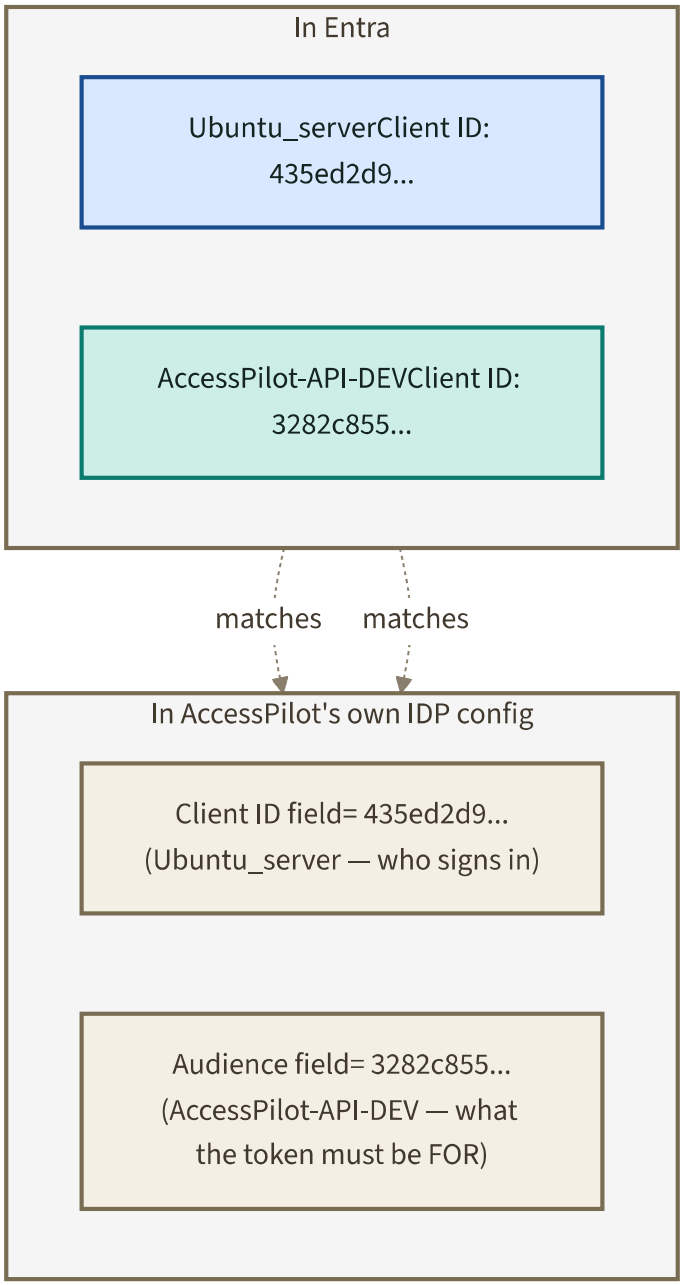
Why App Roles still had to be assigned separately

Granting the permission only answers "*can* Ubuntu_server ask for this kind of token" — it says nothing about *which people* get which role once they have one. That's a completely separate step: **Enterprise applications → AccessPilot-API-DEV → Users and groups**, assigning Test-Admin to `AccessPilot.User` / `.Admin`. Skip that, and you get the right `aud` and `scp` but an empty `roles` claim — a different failure, same symptom (401).

Why AccessPilot's own config had to point at AccessPilot-API-DEV too

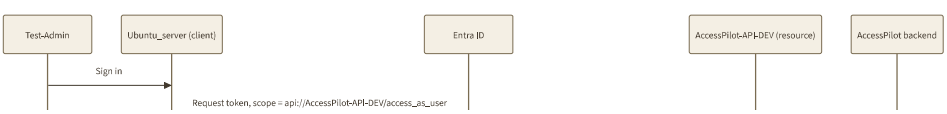
Granting the Entra-side permission gets the right token *issued*. But AccessPilot's backend still has to know what a "correct" token is supposed to look like, so it can check an incoming one against that expectation. That's what the **Audience** field in AccessPilot's own IDP configuration is for.

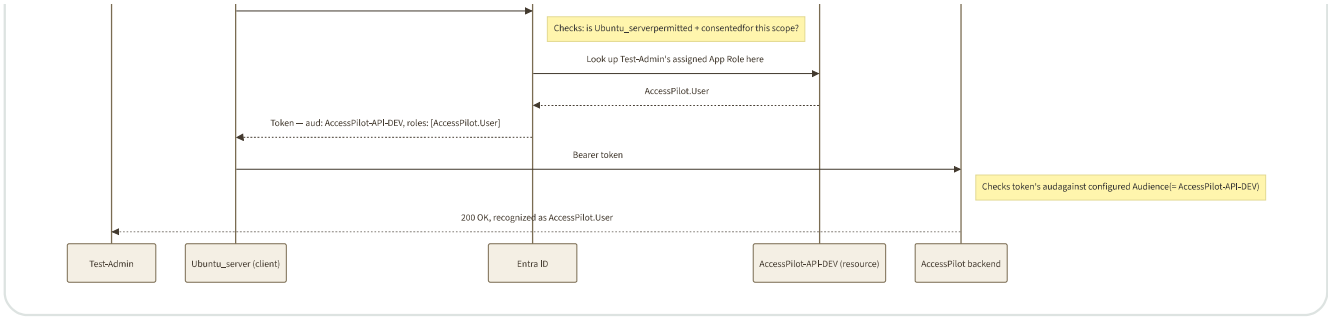
WHERE EACH SIDE'S CONFIGURATION POINTS



If **Audience** is left blank, AccessPilot quietly assumes the resource app and the client app are the *same* registration (a very common, simpler setup) and checks incoming tokens against the Client ID instead. Since yours are two different apps, that assumption is wrong — the real token's `aud` (AccessPilot-API-DEV) would never match what AccessPilot was checking for (Ubuntu_server), and every request would 401 even with Entra configured perfectly.

Putting the whole chain together





Three separate places had to agree on "AccessPilot-API-DEV is the resource" — the permission grant (Entra lets the client ask for it), the role assignment (Entra decides who gets what once they have a token for it), and the Audience field (AccessPilot itself knows to expect that identity in the token). Any one of the three being wrong produces the exact same 401 with no obvious clue which — which is exactly what made this one slow to track down live.